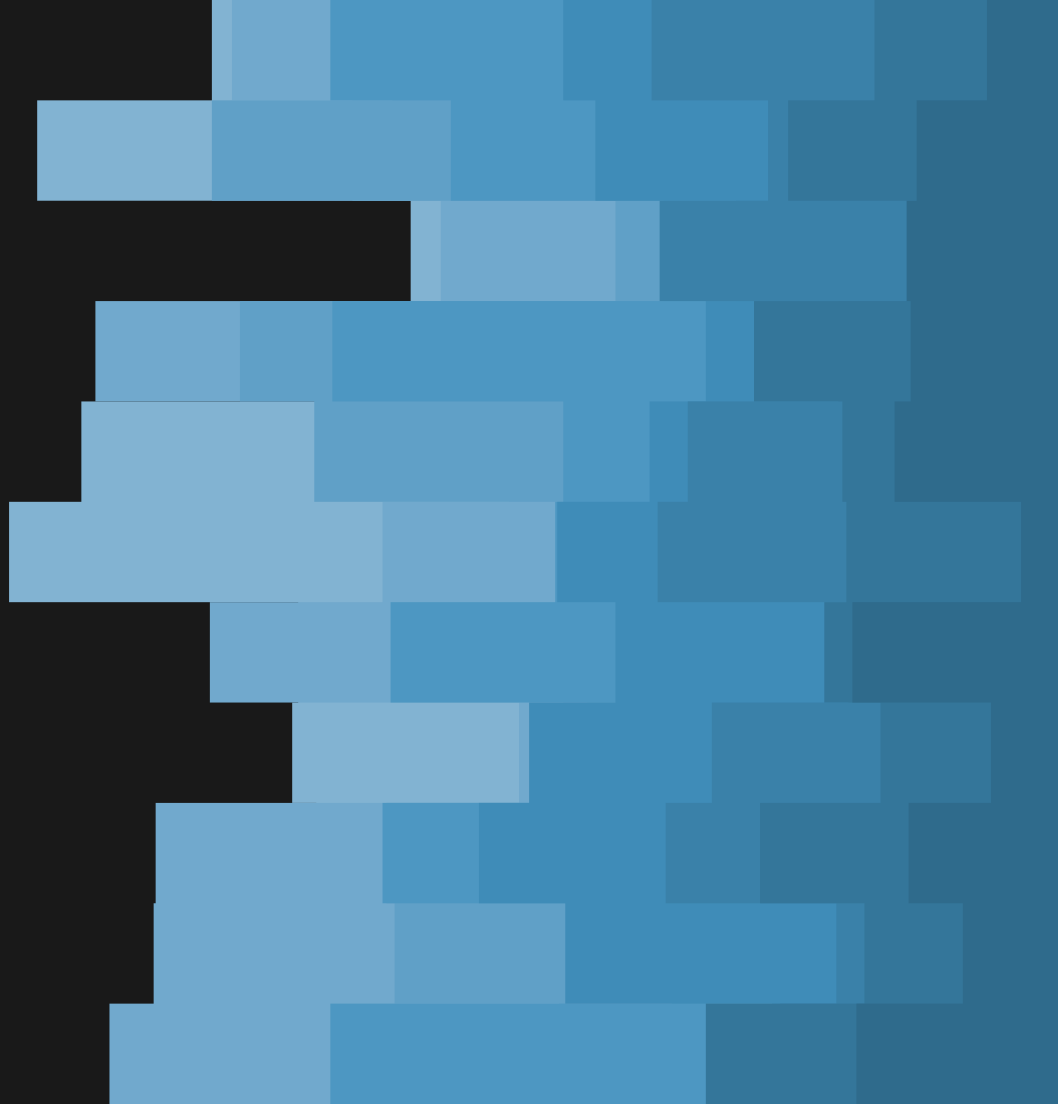
JSX

JavaScript XML for React



O que é JSX?

JavaScript XML

- Permite escrever sintaxe HTML diretamente em JavaScript
- Facilita a escrita e adição de HTML no React
- É traduzido para JavaScript em tempo de execução
- Baseado na extensão ES6

Por que JSX?

Limpo e melhor legibilidade

JSX

```
1  const myElement = <h1>I Love JSX!</h1>;
2
3  createRoot(document.getElementById('root')).render(
4    myElement
```

JS

```
1  const myElement = React.createElement('h1', {}, 'I do not use JSX!'
2
3  createRoot(document.getElementById('root')).render(
4    myElement
```

JSX is HTML-like

JSX converte tags HTML em elementos React

```
1  const myElement = <h1>Hello React!</h1>;
2
3  createRoot(document.getElementById('root')).render(myElement);
```

- Não necessita `createElement()` ou `appendChild()`
- HTML inserido diretamente no JS

Expressões em JSX

Uso de `{ }`

```
1  const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
```

Expressões válidas:

- Variáveis
- Propriedades
- Expressões JS

HTML Multi linhas

Uso de `( )`

```
const myElement = (
  <ul>
    <li>Apples</li>
    <li>Bananas</li>
    <li>Cherries</li>
  </ul>
);
```

One Top Level Element

Tag única

```
1  // - Correct - wrapped in div
2  const myElement = (
3    <div>
4      <p>I am a paragraph.</p>
5      <p>I am a paragraph too.</p>
6    </div>
7  );
```

> Sem um elemento pai, o JSX gera um erro!

Fragmentos

Uso de `<></>`

```
1  const myElement = (
2    <>
3      <p>I am a paragraph.</p>
4      <p>I am a paragraph too.</p>
5    </>
6  );
```

> Alternativa mais limpa do que utilizar uma `<div>` extra

Elementos Fechados

Fechamento obrigatório de tags

```
1  // - Correct
2  const myElement = <input type="text" />;
3
4  // ✗ Wrong
5  const myElement = <input type="text">;
```

> JSX gera um erro se os elementos não forem fechados corretamente

`className`

- O atributo `class` é uma palavra reservada JS

```
1  const myElement = <h1 className="myclass">Hello World</h1>;
```

> JSX traduz `className` em atributos `class` quando renderizado.

Comentários

`{/* */}`:

```
1  const myElement = <h1>Hello {/* Wonderful */} World</h1>;
```

> Os comentários devem estar dentro de chaves no JSX.

Componentes

JSX em Componentes

- O React usa componentes para construir interfaces de usuário
- Os componentes são trechos de código independentes e reutilizáveis
- São como funções JavaScript e retornam HTML
- O JSX funciona perfeitamente dentro dos componentes do React

JSX em Componentes

Exemplo

```
1  function Car() {
2    return (
3      <>
4        <h2>My Car</h2>
5        <p>It is a Ford Mustang.</p>
6      </>
7    );
8  }
9
10 createRoot(document.getElementById('root')).render(<Car />);
```

JSX em Componentes

Realizando operações

```
1  function Car() {
2    const brand = "Ford";
3    const model = "Mustang";
4
5    return (
6      <>
7        <h2>My Car</h2>
8        <p>It is a {brand} {model}.</p>
9      </>
10   );
11 }
```

Resumo JSX

- JSX torna o código React mais legível e natural
- JSX parece HTML, mas na verdade é JavaScript
- Expressões devem ser colocadas dentro de chaves `{ }`
- Sempre envolva JSX com várias linhas entre parênteses
- Use um elemento (ou fragmentos) de nível superior
- Todos os elementos devem ser autofechados ou fechados corretamente
- Use `className` em vez de `class`
- JSX funciona perfeitamente em componentes React

Expressões JSX

> JSX permite incorporar **qualquer expressão JavaScript válida** dentro da marcação.

- Envolva expressões em chaves `{ }`
- O React avalia a expressão e renderiza o resultado
- Torna o JSX dinâmico e poderoso

Expressões JSX

Exemplo

```
1  function Car() {
2    return (
3      <>
4        <h1>My car</h1>
5        <p>It has {218 * 1.36} horsepower</p>
6      </>
7    );
8  }
```

Usando Variáveis

```
1  function Car() {
2    const hp = 218 * 1.36;
3    return (
4      <>
5        <h1>My car</h1>
6        <p>It has {hp} horsepower</p>
7      </>
8    );
9  }
```

Defina variáveis fora do JSX e faça referência a elas dentro de `{ }`.

Chamadas de Função

```
1  function kwtohp(kw) {
2    return kw * 1.36;
3  }
4
5  function Car() {
6    return (
7      <>
8        <h1>My car</h1>
9        <p>It has {kwtohp(218)} horsepower</p>
10     </>
11   );
12 }
```

> As funções retornam valores que são renderizados.

Propriedades de Objetos

dot notation

```
1   function Car() {
2     const myobj = {
3       name: "Fiat",
4       model: "500",
5       color: "white"
6     };
7
8     return (
9       <>
10        <h1>My car is a {myobj.color} {myobj.name} {myobj.model}</h1>
11      </>
12    );
13  }
```

> Combine várias propriedades em uma única expressão JSX.

Expressões em JSX

Resumo

- Qualquer expressão JavaScript válida funciona
- Deve estar entre chaves `{ }`
- É avaliada no momento da renderização
- Pode incluir variáveis, chamadas de função e propriedades de objetos
- O resultado é inserido no DOM

Expressões em JSX

Usos típicos

- Cálculos matemáticos
- Concatenação de strings
- Renderização condicional (tópicos posteriores)
- Mapeamento de arrays (tópicos posteriores)
- Formatação de valores
- Geração de conteúdo dinâmico

Atributos JSX

`class` versus `className`

> In JSX, `class` is a reserved JavaScript keyword

```
function Car() {
  return (
    <h1 className="myclass">Hello World</h1>
  );
}
```

Expressões como atributos

> Pode-se utilizar expressões JS para valores de atributos dinâmicos

```
1  function Car() {
2    const x = "myclass";
3    return (
4      <h1 className={x}>Hello World</h1>
5    );
6  }
```

- Envolver expressões em chaves `{ }`
- Não utilizar aspas
- Perfeito para estilização e comportamento dinâmicos

Atributos de Eventos em *camelCase*

```
1  function Car() {
2    const myfunc = () => {
3      alert('Hello World');
4    };
5
6    return (
7      <button onClick={myfunc}>Click me</button>
8    );
9  }
```

- `onClick` ao invés de `onclick`
- `onMouseOver` ao invés de `onmouseover`
- Segue a convenção de nomes do JS

Atributos Boleanos

Diferente em JSX

```
1  // This makes the button disabled
2  <button onClick={myfunc} disabled>Click me</button>
3
4  // Also disabled
5  <button onClick={myfunc} disabled={true}>Click me</button>
6
7  // NOT disabled
8  <button onClick={myfunc} disabled={false}>Click me</button>
```

> Presença do atributo implica `true`

Atributo `style`

> Atributos de estilo aceitam objetos JavaScript com propriedades em camelCase

```
1   function Car() {
2     const mystyles = {
3       color: "red",     fontSize: "20px",     backgroundColor: "lightyellow",
4     };
5     return (
6       <>
7         <h1 style={mystyles}>My car</h1>
8       </>
9     );
10  }
```

- Objeto com propriedades CSS em camelCase

Regras de estilo para objetos

Diferenças entre HTML e CSS

- Propriedades em camelCase
- Valores como *strings*
- Sem seletores CSS (apenas estilos inline)
- Permite o uso de variáveis e expressões

```
1  const buttonStyle = {
2    backgroundColor: 'blue',
3    color: 'white',
4    padding: '10px 20px',
5    borderRadius: '5px'
6  };
```

Padrões Comuns para Atributos

- `className` para classes CSS
- `onClick`, `onChange`, etc para eventos
- `style` para estilos *inline*
- `disabled`, `checked` para boleanos
- Usar `{ }` para valores dinâmicos

Condicionais

Renderização Condicional em JSX

> React não permite instruções `if` diretamente dentro do JSX.

Há duas abordagens a seguir:

1. Escrever instruções `if` fora do JSX
2. Usar expressões ternárias dentro do JSX

Renderização Condicional em JSX

`if` fora do JSX

> Utilizar a lógica antes de `return`

```
function Fruit() {
  const x = 5;
  let y = "Apple";
  if (x < 10) {
    y = "Banana";
  }
  return (
    <h1>{y}</h1>
  );
}
```

- A lógica ocorre antes do JSX
- A variável contém o resultado

Renderização Condicional em JSX

Expressões ternárias

```
1  function Fruit() {
2    const x = 5;
3
4    return (
5      <h1>{(x) < 10 ? "Banana" : "Apple"}</h1>
6    );
7  }
```

- Condição ? valor_verdadeiro : valor_falso
- Deve estar entre chaves `{ }`
- Perfeito para condições simples

Operador Ternário

Sintaxe

```
1  condition ? expression_if_true : expression_if_false
```

- `condition` : expressão boleana
- `?` : Executado quando `true`
- `:` : Executado quando `false`

Quando Usar Cada Abordagem?

`if` fora do JSX

- A lógica for complexa
- Houver múltiplas condições
- For necessário definir múltiplas variáveis
- Estrutura de componente mais limpa

Expressões ternárias

- Houver condições simples de verdadeiro/falso
- Houver renderização condicional embutida
- Houver expressões curtas e legíveis

Outros Casos

Múltiplos `if-else`

```
1  function StatusMessage({ status }) {
2    let message;
3
4    if (status === 'loading') {
5      message = 'Loading...';
6    } else if (status === 'success') {
7      message = 'Success!';
8    } else {
9      message = 'Error occurred';
10   }
11
12   return <p>{message}</p>;
13 }
```

Outros Casos

Ternários Aninhados

```
// Readable
{isLoggedIn ? 'Welcome!' : 'Please log in'}

// Less readable - avoid if possible
{status === 'loading' ? 'Loading...' :
 status === 'success' ? 'Done!' : 'Error'}
```

Boas Práticas

- Mantenha as condições simples e legíveis
- Use instruções `if` para lógica complexa
- Use operadores ternários para condições simples em linha
- Considere extrair lógica complexa para funções
- Teste todos os ramos condicionais

Resumo

Condicionais

> A renderização condicional torna os componentes dinâmicos

- As instruções `if` devem estar fora do JSX
- Os operadores ternários funcionam dentro de `{ }`
- Escolha a abordagem com base na complexidade
- Sempre teste os casos verdadeiro e falso
- Mantenha o código legível e de fácil manutenção

Perguntas

Exercícios

1

Realizar o Quiz

Referências

- Introdução JSX
- React

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

Powered by Slidev

Cover image by haikei